2023 Latin American Robotics Symposium (LARS), 2023 Brazilian Symposium on Robotics (SBR), and 2023 Workshop on Robotics in Education (WRE) | 979-8-3503-1538-7/23/$31.00 ©2023 IEEE | DOI: 10.1109/LARS/SBR/WRE59448.2023.10333068

Dataset and Baseline Experiments for Self-Localization and Tracking in the RoboCup Small Size League

João G. Melo, Lucas Cavalcanti, Riei Joaquim, Victor Araújo and Edna Barros
Centro de Informática
Universidade Federal de Pernambuco
Recife, Pernambuco, Brazil
{jgocm, lhcs, rjmr, vhssa, ensb}@cin.ufpe.br

***Abstract*—The RoboCup Small Size League (SSL) and the Vision Blackout challenge, aim to increase robot autonomy and encourage teams to explore local sensing and processing. Thus, this paper proposes the use of an open-source dataset for the self-localization and tracking of SSL robots, which was recorded onboard and contains ground-truth poses, estimated by ssl-vision, odometry and speed data, computed onboard. Recordings contain nine different scenarios, with multiple paths and velocities, and goals' detections were annotated. A Monte Carlo Localization (MCL) algorithm implementation is described and tested on the proposed dataset, and evaluation metrics and experiment results from the self-localization and odometry algorithms using the collected data are presented. This work aims to enable SSL teams to explore and evaluate solutions for robot self-localization and tracking problems, providing baseline experiments and metrics for objective comparisons of future solutions.**

I. Introduction

Intending to aim for faster and more complex games, the RoboCup Small Size League (SSL) has been advancing toward increasing the robot's autonomy, motivated mainly by the introduction of the Vision Blackout technical challenge [1]. This challenge focuses on demonstrating standard SSL soccer skills (e.g. locomotion, ball manipulation, and collaboration) without the currently shared vision software, SSL-Vision [2], encouraging teams to explore local sensing and processing rather than an off-board computer and a global set of cameras sensing the environment.

For the Vision Blackout challenge, SSL robots, usually constrained to 180 and 150 millimeters in diameter and height, respectively, do not have height restrictions, enabling teams to add computing modules and onboard cameras to the robot. Following this approach, solutions for autonomously executing autonomous skills have been proposed in recent works [3], [4]. Although these authors presented a better perception of robots, none proposed solutions for embedded self-localization, which is critical for developing more onboard skills.

Typical strategies for addressing the self-localization problem at other RoboCup leagues involve using Monte Carlo Localization (MCL) [5], [6]: a particle filter-based algorithm for regressing a robot's position and orientation (pose) in a previously mapped environment [7]. This technique requires the robot to acquire information about the environment using cameras or LiDAR sensors, for instance, and estimate its motion based on its actions, with inertial sensors, for example.

So far, only a few teams in the SSL have been able to implement the necessary hardware adaptations for building a fully autonomous robot, reducing the possibilities of solutions being proposed for the self-localization problem. Also, disposing of an SSL field with similar dimensions to the RoboCup standards is essential for making reliable tests. Therefore, an open-source dataset containing all necessary data for an MCL algorithm implementation would enable SSL teams to propose and evaluate solutions for the self-localization problem.

Using common datasets and evaluation metrics have shown to significantly help push the state-of-the-art forward at highly relevant problems in robotics, such as the Simultaneous Localization and Mapping (SLAM) [8]. It enables researchers to focus on developing different algorithms and allows for more reliable evaluations and objective comparisons since the same metrics and ground-truth data are used. Also, it suggests what sensors and information are needed for implementing the desired algorithm.

This paper provides detailed descriptions of an open-source onboard recorded dataset, with the goal of enabling SSL teams to explore and evaluate solutions for the robot's self-localization and tracking problems. It contains data from nine recordings, with multiple paths and velocities, including images captured by the robot's on-board camera, ground-truth poses, estimated by SSL-vision, odometry, and speed data, computed onboard with an STM32F767ZI Microcontroller Unit (MCU) using techniques presented in [4]. A 4GB NVIDIA Jetson Nano Developer Kit was employed for embedded vision processing and log recording.

For demonstrating the recordings' suitability for use in evaluating the performance of self-localization solutions, we present a case study in Section V, proposing observation,

979-8-3503-1538-7/23/$31.00©2023IEEE

https://github.com/jgocm/ssl-navigation-dataset

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:30:26 UTC from IEEE Xplore. Restrictions apply.

motion and resampling models for implementing an MCL algorithm on the given dataset. Section VI presents evaluation metrics and resulting accuracies from self-localization and odometry using the collected data, and we consider that the main contributions of this paper are:

- A public dataset for evaluating self-localization and tracking algorithms.
- Introducing models for an MCL algorithm implementation in the SSL environment.
- Baseline experiments and evaluation metrics for objective comparisons of different algorithms.

II. Related Work

The Vision Blackout challenge [1] aims at solving four basic tasks: grabbing a stationary ball (I), scoring on an empty goal (II), moving to specific coordinates (III), and scoring an indirect goal (passing to another robot) (IV). So far, solutions for detecting SSL objects and computing their relative positions have been proposed [3], [9], and, by using this information for making decisions and acting, teams were only capable of completing stages I and II.

Performing self-localization is a must for solving the challenge's stage III and may also bring performance improvements at other tasks, as observed by [4]. They propose a complete SSL robot architecture for executing basic skills autonomously, using encoder and gyroscope readings to compute the robot's odometry, and evaluate the system's performance in three different tasks: grabbing a ball, scoring on an empty goal and passing the ball. Results show that planning more efficient paths, discarding out-of-field information, avoiding penalties, and making more efficient field explorations are some of the needed improvements for the system. The authors suggest that self-localization information might be the key to surpassing these limitations.

Monte Carlo Localization (MCL) is a particle filter-based technique that uses sensor observations and movement updates for estimating a robot's pose in a known environment [7]. Its use was initially proposed in the Sony Four-Legged Robot League (SFRL), employing onboard sensing and processing. Even for highly constrained hardware and unreliable measurements, the algorithm was capable of solving the localization problem, mainly using field lines, boundaries and landmarks for observation updates, and rough estimates of the robot's motion for movement updates [10].

Particle filter-based approaches are still employed in RoboCup Leagues, showing their robustness for solving the self-localization problem [5], [6]. However, its reproduction at the SSL environment requires adaptations and implementing more onboard features, such as field elements detection (e.g., lines, borders, goal posts, corners, and field center), demanding onboard-recorded data for testing and evaluating solutions.

To surpass hardware and environment infrastructure limitations and allow testing and tuning under various conditions, the European Robotics Challenge (EuRoC) proposes using common datasets [11]. They dispose of onboard-recorded data for visual-inertial SLAM and 3D reconstruction evaluation

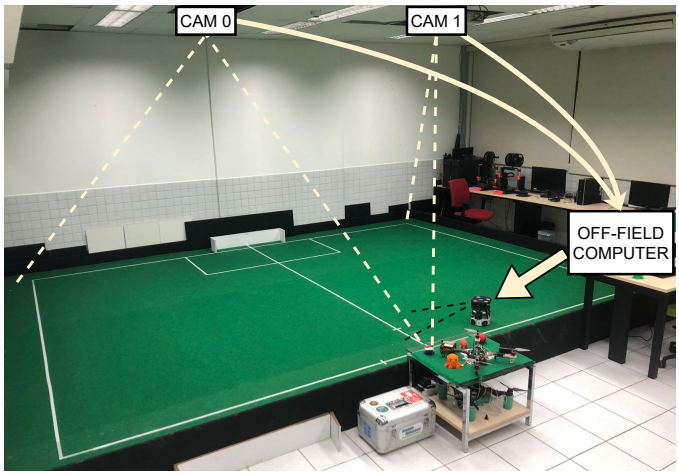


Fig. 1. Recordings environment with a 6m x 4.5m SSL field.

in 11 different scenarios, with varying environments, paths, velocities, and duration, enabling teams to focus on developing more efficient algorithms.

Similarly to the EuRoC, for evaluating SLAM and odometry systems based on RGB-D data, another research presents datasets recorded from a wheeled mobile robot and splits them into training and testing parts [8]. They also propose evaluation metrics and provide automatic evaluation tools, allowing for more reliable and objective performance comparisons on different algorithms. The proposed metrics are based on computing the Root Mean Squared Error (RMSE) over all time indices of translational components from the robot's pose.

III. Data Acquisition and Processing

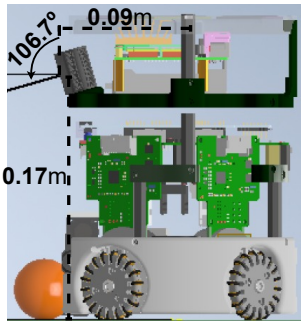
Recordings were conducted using an SSL robot with additional hardware for embedded vision processing and data storage. The robot's ground-truth position was acquired with SSL-vision, and an off-field computer was used for sending messages to the robot, as illustrated in Figure 1. The following subsections detail the utilized hardware and field configurations, and the procedures and techniques adopted for data recording and processing.

A. Onboard Data

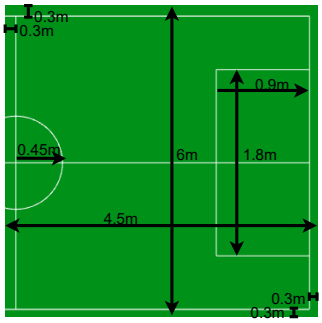
Onboard navigation data was recorded using the hardware proposed in [4], which employs an STM32F767ZI Microcontroller Unit (MCU) for communicating with the off-field computer and controlling 4 Maxon EC-45 flat - 50W motors, using 2-channel 1024 steps incremental encoders for velocity feedback. Also, it combines information from encoder readings with gyroscope measurements from an MPU-6050 Inertial Measurement Unit (IMU), for estimating the robot's odometry by updating its current speeds and movements at a 200Hz rate.

Embedded vision data was acquired with a Logitech C922 camera using 640x480 resolution and 30Hz capture rate, with its intrinsic parameters previously calibrated [9]. Extrinsic parameters were defined from the robot's dimensions in Figure

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:30:26 UTC from IEEE Xplore. Restrictions apply.



(a) SSL robot with camera translation and rotation.



(b) Field dimensions and configuration.

Fig. 2. Employed robot and field dimensions.

2. This visual information is processed and stored by a 4GB NVIDIA Jetson Nano. Timestamps from the captured frames are also annotated and used for measuring the recordings' average processing rate, as shown in Table II, resulting in an overall 21.24 frames per second (FPS).

B. Field and Ground-Truth Data

The dataset was recorded on a 6m x 4.5m, positive-half SSL field, as shown in Figure 2. The robot's ground-truth poses were estimated by SSL-vision [2], with frames being captured at a 30Hz rate and sent to an off-field computer, which sent movement commands and ground-truth poses to the MCU through radio frequency wireless communication.

We use embedded odometry information to estimate the robot's displacements between SSL-vision updates due to the low frame rates and the latency for delivering ground-truth information to the MCU. This data is then sent to the Jetson Nano along with speed and odometry data through an Ethernet connection using User Datagram Protocol (UDP) Socket packets.

C. Data Post-Processing

After the acquisition, we employ the objects detection approach from [9] for annotating goals' bounding boxes, as it can be used as a reference for mirrored pose corrections during self-localization. This procedure was executed in the Jetson Nano with a 0.3 score threshold.

The object detection model sporadically makes false-positive inferences. Therefore, we also annotate if the goal is inside the onboard camera's field-of-view (FOV) based on the robot's ground-truth pose, allowing for detection corrections. This analysis was conducted offboard.

Limitations from our software architecture cause the robot only to receive position updates when it starts moving. Thus, in the post-processed log, we removed data from initial frames since their ground-truth information was unreliable. Both the raw and the post-processed data are available in the dataset.

IV. Dataset and Files Organization

A. Dataset

We split the dataset into three types of paths: squared (sqr), random (rnd), and in-game situations (igs). For each path type, we set the robot's maximum speed to 1 (labeled as 01), 1.5 (02), and 2 m/s (03), resulting in 9 scenarios with different average speeds, duration, and odometry accuracies, all presented in Table II.

Squared Paths. The robot performs three laps of 5m x 2.5m rectangular paths in this scenario, mainly with forward and clockwise rotational moves. As a result, the robot rarely sees or detects the goal during the trajectories, and reaches higher velocities due to distances between points being further.

Random Paths. The robot navigates through 25 random poses on the field, combining linear and angular movements. At the start, the robot looks towards the goal, allowing for corrections of mirrored pose estimations. In contrast, inertial odometry tracking accumulates more errors, due to the longer durations and the combinations of translation and rotational movements.

In-Game Situation Paths. This scenario reproduces a similar robot soccer game situation to the Vision Blackout challenge stage II: scoring on an empty goal. The robot is positioned at the field's corner, and the ball in the opposite half. In this scenario, three movements are executed: rotating on its axis to look towards the ball, moving to the goal-to-ball line projection looking towards the ball, and moving forward to kick it to the goal.

B. Data Formats and Organization

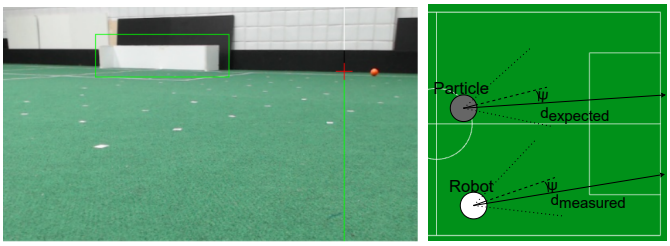
Data from recordings are stored in the "data" folder with subfolders for the different scenarios. The "cam" folder contains camera frames, and "logs" folder includes ".csv" files corresponding to the raw and post-processed navigation data. The "configs" folder contains ".txt" files for the camera's parameters and field dimensions. In contrast, the "src" folder holds base classes for implementing Python scripts, e.g., for making evaluations or simulations.

C. CSV Data Fields

Fields from the ".csv" files are:

- "FRAME_NR": indicates which onboard frame the data corresponds to.
- "ODOMETRY X/Y/THETA": total odometry in meters and radians.
- "POSITION X/Y/THETA": robot ground-truth pose in meters and radians.
- "SPEED X/Y/W": current robot speeds in m/s and rad/s.
- "GOAL IN FOV": indicates if the goal is inside the robot's field of view, according to its ground-truth position in the field.
- "HAS GOAL": indicates if a goal was detected by the objects detector.
- "X/Y_MIN/MAX": detected goal bounding box coordinates.
- "TIMESTAMP": the timestamp from the onboard frame capture in seconds.

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:30:26 UTC from IEEE Xplore. Restrictions apply.



(a) Resulting frame after vision processing. (b) Robot and particle visions.

Fig. 3. Observations with onboard vision on a SSL field for a MCL algorithm.

V. Case Study: Dataset Utilization

This Section provides details from an MCL algorithm implementation for estimating the robot's pose on the given dataset, explaining its general workflow and particularities for the SSL environment, demonstrating how the presented data can be used for exploring self-localization solutions.

A. The General Monte Carlo Localization

MCL maintains a set of N samples for approximating a Probability Density Function (PDF) that represents the robot's state, x_t [7]. At each time step, the particle states, x_t^m, are updated based on the robot's motion model. Particles weights, w_t^m, are then updated based on similarities between the sensor measurements, z_t, and their expected observations given the particle state, being expressed as a conditional probability $p(z_t|x_t^m)$. Finally, if needed, particles are resampled accordingly to their weights to generate a new set for the next iteration, and this process is repeated iteratively.

B. Field Boundary Detection

In the SSL, three main classes of field elements can be used as references for localization: goals, field markings (lines), and boundaries. Goals can be detected using the approach from [9]. We tested multiple approaches for detecting field lines and boundaries using adaptations of grid-based line detection techniques from other RoboCup Leagues [5], [10]. However, the ones for lines did not achieve fast and robust results when running on the Jetson Nano; thus, we only use field boundaries for self-localization, performing vertical line scans.

A bottom-up scan on a vertical line is performed by making color segmentation and checking whether a green-to-black transition exists; if yes, we check if the ten subsequent pixels are also black, and if yes, the first pixel of the sequence is taken as the field boundary. Figure 3 illustrates the resulting frame after performing a goal detection and a vertical line scan, highlighting a boundary point with a red cross mark. Offline tests in the Jetson Nano have shown that each vertical line scan takes approximately 8 ms to process, so we opted for using only one randomly positioned line at each frame, avoiding positions inside detected bounding boxes.

C. Observation Model

We define observation models for two types of detections: goal and field boundary. Boundaries can be seen from almost any location on the field and provide information about both Cartesian directions; however, the field is symmetric regarding them, which may cause mirrored pose estimations. On the other hand, goals are rarely seen from the robot's perspective but give orientation information, enabling to correct mirroring issues on a half-field setup.

1) Boundary Points Observation: Field boundary XY relative coordinates to the robot are computed using camera parameters by assuming they lay on the ground, as in [9], and converted to relative distance, d_j, and angle, ψ_j. For each sample, expected distance $\hat{d}_j^m$ is computed by projecting a line from the particle's position towards the measured ψ_j direction, and finding the first intersection with a field border, as shown in Figure 3.

Similarities are computed assuming a Gaussian distribution, such as the Landmark Observation from [6], but for $\psi_j = \hat{\psi}_j^m$. However, distance measurement deviations are higher for further points [9], [10]. Considering that standard deviation, σ_d, is proportional to d_j, probabilities are estimated by Equation 1, where α can be tuned to change the influence of boundary detections on determining the robot's pose.

$$p(z_t|x_t^m) = exp\left[-\alpha\left(1 - \frac{\hat{d}_j^m}{d_j}\right)^2\right]. \quad (1)$$

2) Goal Observation: Similarities from goal observations are expressed as booleans, i.e., 1 or 0. The particle's FOV is projected onto the field, as in Figure 3, determining whether a goal is inside its vision range. If the robot detects a goal and the particle's vision does not cover it, the similarity is set to 0; otherwise, it is 1.

D. Motion Model

Robot's movements are estimated by the embedded inertial odometry [4], expressed as $\Delta d_t = [\Delta x_t, \Delta y_t, \Delta\theta]$. As the technique is prone to deviations, estimations are added to a Gaussian error. Samples' poses are updated according to $x_t^m = x_{t-1}^m + \mathcal{N}(\Delta d_t, \sigma_{\Delta d})$, where $+$ operations involve coordinates rotation to the robot's axis, and standard deviations are proportional to the distance travelled or rotation performed, $\sigma_{\Delta d} = [\gamma_x \Delta x_t, \gamma_x \Delta y_t, \gamma_\theta \Delta\theta]$.

E. Resampling

A systematic resampling [12] is applied to the distribution whenever the robot has travelled or rotated for more than 1 meter or 45 degrees since the last resampling or one particle has its normalized weight higher than 0.9. Also, random uniform deviations are added to the resampled particles, where the less probable ones are moved more, as presented in [10], with the maximum deviations denoted as $[\Delta_{trans}, \Delta_{trans}, \Delta_{rot}]$.

VI. Experiments, Metrics and Results

We demonstrate the dataset's suitability for the self-localization problem by running an MCL algorithm, with the particularities presented in Section V, in the proposed data. For accuracy measuring, similar to the Relative Pose Error

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:30:26 UTC from IEEE Xplore. Restrictions apply.

(RPE) from [8], we compute the entire trajectory's Root Mean Square Error (RMSE) from translational components. Also, to illustrate our results, we plot ground truth, inertial odometry, and MCL-estimated trajectories and position errors over time for three scenarios: *rnd_01*, *sqr_02* and *igs_03*.

A. Experimental Setup

We present our inertial odometry's trajectory by setting its initial state to the robot's ground-truth pose, applying the motion model rule from Subsection V-D for updating movements, except that we do not add any noises.

For the self-localization algorithm, the N particles are initialized inside a 0.5 meters radius around a position seed, with uniformly distributed poses, according to the robot's initial state, following the Vision Blackout challenge rules [1]. The parameters utilized during experiments are shown in Table I, following the approach presented in Section V.

TABLE I
PARAMETERS FOR THE MCL ALGORITHM.

Parameter	N	α	γ_x	γ_y	γ_θ	Δ_{trans}	Δ_{rot}
Value	100	10	1	1	0.1	0.2m	15º

B. Evaluation Metrics

MCL generates a set of particles representing the robot's state, and we measure its accuracy based on the particles' poses weighted sum. Root Mean Square Error (RMSE) is employed for evaluating trajectory errors from $RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\|\boldsymbol{\delta_i} - \hat{\boldsymbol{\delta}}_{\boldsymbol{i}}\|^2}$, on which n accounts for the number of trajectory points. $\boldsymbol{\delta_i}$ and $\hat{\boldsymbol{\delta}}_{\boldsymbol{i}}$ indicate the calculated and ground-truth positions, respectively.

For qualitative analysis, we observe the algorithms' behaviors along the paths, plotting Euclidean distances over time and trajectory comparisons to the ground truth for odometry and self-localization. During random paths, the robot's movements exhibit considerable overlaps, rendering them challenging to visualize; therefore, only the 800 first steps are presented in the plots.

C. Results

Overall characteristics of the resulting dataset are shown in Table II, with last two columns presenting errors from odometry and MCL (lower is better), and Figure 4 presents examples of images onboard images captured in the recordings. We observe that the self-localization algorithm was able to maintain the robot's pose on all recordings, indicating its robustness towards varied paths and demonstrating the recorded data's reliability. In contrast, the odometry accumulates errors over time, resulting in higher RMSEs. Also, higher robot speeds cause measurements to be less accurate; thus, faster scenarios tend to result in lower errors.

Figure 5 presents ground truth (red), MCL (blue dashed), and odometry (green dotted) trajectories, with coordinates in meters. It highlights that MCL's particles tend to spread when the robot is far from the borders; due to the observations becoming less accurate for higher distances. Even so, the self-localization algorithm was able to recover the robot's tracking, reducing its distance to the ground truth, as illustrated in Figure 6, which shows the odometry (green) and MCL (blue dashed) distances to ground truth over time, in meters and seconds.

TABLE II
DATASET CHARACTERISTICS FROM POST-PROCESSED DATA AND EVALUATION RESULTS

Name	**Duration (s)**	**Avg. Trans. Vel. (m/s)**	**Avg. Rot. Vel. (deg/s)**	**FPS (Hz)**	**Odometry RMSE**	**MCL RMSE**
sqr_01	97.29	0.469	0.250	22.22	0.472	0.274
sqr_02	81.37	0.567	0.290	22.05	0.468	0.285
sqr_03	70.98	0.662	0.342	20.66	1.245	0.510
rnd_01	114.21	0.423	0.367	21.27	0.476	0.414
rnd_02	101.96	0.531	0.456	19.94	0.945	0.442
rnd_03	95.12	0.589	0.464	21.10	0.911	0.547
igs_01	25.40	0.325	0.268	21.37	0.284	0.221
igs_02	25.09	0.395	0.255	21.87	0.420	0.241
igs_03	25.33	0.382	0.274	21.59	0.258	0.196

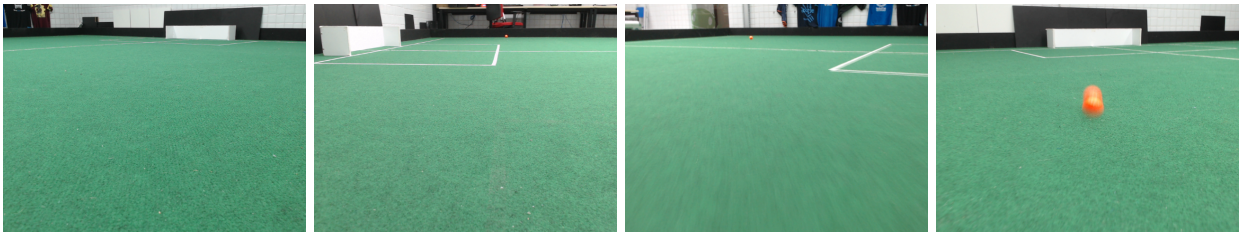
VII. CONCLUSION AND FUTURE WORK

This work proposes the usage of a public dataset for exploring and evaluating robot self-localization and tracking algorithms in the SSL. The open-source data recorded onboard an SSL robot on a 6m x 4.5m half-field configuration,are presented and have its use demonstrated with a Monte Carlo Localization algorithm implementation. The particularities of the implemented MCL are explained, and experiments show that varied paths and moves have different impacts on self-localization and tracking, as the inertial odometry achieves better accuracy for one-dimensional movements and camera detections are more precise for nearer points. Thus, the multiple scenarios allow for testing and evaluation under various conditions, and objective comparisons can be realized by using the presented metrics and baseline results.

Ground-truth data was acquired using SSL-vision with cameras at 30Hz frame rates, which is low compared to RoboCup SSL standards. Also, a reduced-size field was employed, configured as a half-side, avoiding estimations of mirrored poses. To overcome these limitations, we plan on expanding the presented dataset in the RoboCup environment, using a complete field configuration and high-standards cameras set, allowing for more reliable algorithm evaluations.

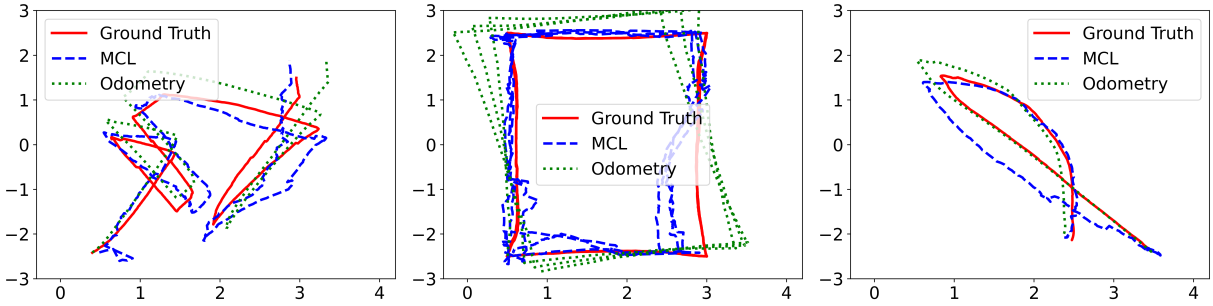
Due to the lower accuracy of further measurements, the implemented MCL algorithm is prone to particles' divergence when the robot is far from the field boundaries, as discussed in Subsection VI-C. Therefore, efficient techniques for onboard field line detection must be developed, since the robot is usually nearer to them. Also, since the goal is to deploy the algorithm to the robot's hardware, solutions for reducing its computational complexity must be investigated, such as using an adaptive number of particles or adding more parallel processing.

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:30:26 UTC from IEEE Xplore. Restrictions apply.



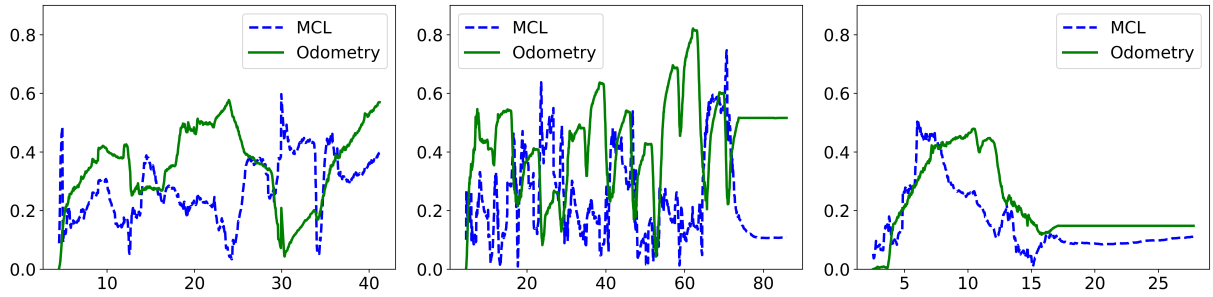
(a) rnd_01. (b) sqr_02. (c) igs_03. (d) igs_03.

Fig. 4. Images from the onboard-recorded dataset in three different scenarios.



(a) *rnd_01*. (b) *sqr_02*. (c) *igs_03*.

Fig. 5. Trajectory comparisons from MCL, odometry and ground-truth in 3 scenarios.



(a) *rnd_01*. (b) *sqr_02*. (c) *igs_03*.

Fig. 6. Distances from MCL and odometry to ground-truth over time in 3 scenarios.

REFERENCES

[1] RoboCup, "RoboCup 2022 SSL Vision Blackout Technical Challenge Rules." [Online]. Available: https://robocup-ssl.github.io/technical-challenge-rules/2022-ssl-vision-blackout-rules.pdf

[2] S. Zickler, T. Laue, O. Birbach, M. Wongphati, and M. Veloso, "Ssl-vision: The shared vision system for the robocup small size league," in *Robot Soccer World Cup*. Springer, 2009, pp. 425–436.

[3] F. Weinmann, "Creating a development and deployment infrastructure for the TIGERs Mannheim on-bot vision software," Baden-Wuerttemberg Cooperative State University Mannheim, Tech. Rep., 05 2021.

[4] J. G. Melo, F. Martins, L. Cavalcanti, R. Fernandes, V. Araújo, R. Joaquim, J. G. Monteiro, and E. Barros, "Towards an autonomous robocup small size league robot," in *2022 Latin American Robotics Symposium (LARS), 2022 Brazilian Symposium on Robotics (SBR), and 2022 Workshop on Robotics in Education (WRE)*, 2022, pp. 1–6.

[5] T. Röfer, T. Laue, A. Baude, J. Blumenkamp, G. Felsch, J. Fiedler, A. Hasselbring, T. Haß, J. Oppermann, P. Reichenberg, N. Schrader, and D. Weiß, "B-Human team report and code release 2019," 2019, only available online: http://www.b-human.de/downloads/publications/2019/CodeRelease2019.pdf.

[6] A. Muzio, L. Aguiar, M. R. Máximo, and S. C. Pinto, "Monte carlo localization with field lines observations for simulated humanoid robotic soccer," in *2016 XIII Latin American Robotics Symposium and IV Brazilian Robotics Symposium (LARS/SBR)*, 2016, pp. 334–339.

[7] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics (Intelligent Robotics and Autonomous Agents)*. The MIT Press, 2005.

[8] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 573–580.

[9] J. G. Melo and E. Barros, "An embedded monocular vision approach for ground-aware objects detection and position estimation," in *RoboCup 2022:*. Springer International Publishing, 2023, pp. 100–111.

[10] T. Röfer and M. Jüngel, "Fast and Robust Edge-Based Localization in the Sony Four-Legged Robot League," in *RoboCup 2003: Robot Soccer World Cup VII*, D. Polani, B. Browning, A. Bonarini, and K. Yoshida, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 262–273.

[11] M. Burri, J. Nikolic, P. Gohl, T. Schneider, J. Rehder, S. Omari, M. Achtelik, and R. Siegwart, "The EuRoC micro aerial vehicle datasets," *The International Journal of Robotics Research*, vol. 35, 01 2016.

[12] T. Li, M. Bolic, and P. M. Djuric, "Resampling methods for particle filtering: Classification, implementation, and strategies," *IEEE Signal Processing Magazine*, vol. 32, no. 3, pp. 70–86, 2015.

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:30:26 UTC from IEEE Xplore. Restrictions apply.